

SMART VEHICLE SAFETY

PHASE 4 — PROTOTYPE DEVELOPMENT REVIEW

(PDR)

Smart Vehicle Accident Detection and Emergency

Rescue System

Objective: convert the approved Phase 3 design into a working integrated prototype and provide reproducible source/evidence.

1. Working Prototype — Version 1

Prototype V1 should integrate ESP32, MPU6050, Neo-6M GPS, buzzer, LED, cancellation switch, battery and buck converter. Validate wiring and sensor operation before final packaging.

Milestone	Evidence
Sensor integration	Stable MPU6050 serial readings
Accident logic	Controlled test triggers expected state
Cancellation	Switch cancels warning
GPS	Valid coordinates with GPS fix
Connectivity	Event reaches cloud/dashboard
Power	Stable operation from selected supply
Physical integration	Secure wiring/enclosure/PCB

2. Embedded Software

The companion source package contains a complete reference firmware file using the proposed ESP32 pins, MPU6050, Neo-6M UART, buzzer, LED, cancellation switch and Firebase HTTP update.

```
#include <Arduino.h>
#include <Wire.h>
#include <WiFi.h>
#include <HTTPClient.h>
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>
#include <TinyGPSPlus.h>

Adafruit_MPU6050 mpu;
TinyGPSPlus gps;
HardwareSerial GPS(2);

const int BUZZER=25, LED=26, CANCEL=27;
const float WARNING_G=3.0, CONFIRMED_G=6.0, TILT_LIMIT=80.0;
const unsigned long WARNING_TIME=60000UL;

const char* WIFI_SSID="YOUR_WIFI";
const char* WIFI_PASSWORD="YOUR_PASSWORD";
const char* FIREBASE_URL=
"https://YOUR_PROJECT-default-rtdb.firebaseio.com/vehicles/DEVICE_ID.json";

enum State { NORMAL, WARNING, ACCIDENT };
State state=NORMAL;
unsigned long warningStarted=0;

float magnitudeG(float x,float y,float z){
    return sqrt(x*x+y*y+z*z)/9.80665;
}
float tiltDeg(float x,float y,float z){
    float h=sqrt(x*x+y*y);
    return atan2(h,fabs(z))*180.0/PI;
}
void alertOn(){digitalWrite(LED,HIGH); tone(BUZZER,2200);}
void alertOff(){digitalWrite(LED,LOW); noTone(BUZZER);}
void readGPS(){while(GPS.available()) gps.encode(GPS.read());}

bool publishEvent(const char* status,float g,float tilt){
    if(WiFi.status()!=WL_CONNECTED) return false;
    String json="{";
    json+="\"status\": \""+String(status)+"\", ";
```

```

    json+="\"impactG\": "+String(g,2)+" ";
    json+="\"tiltDeg\": "+String(tilt,1)+" ";
    json+="\"gpsValid\": "+String(gps.location.isValid())?"true":"false"+" ";
    json+="\"latitude\": "+String(gps.location.isValid()?gps.location.lat():0.0,6)+" ";
    json+="\"longitude\": "+String(gps.location.isValid()?gps.location.lng():0.0,6)+" ";
    json+="\"timestamp\": "+String((unsigned long)millis());
    json+="}";
    HTTPClient http; http.begin(FIREBASE_URL);
    http.addHeader("Content-Type", "application/json");
    int code=http.PUT(json); http.end();
    return code>=200 && code<300;
}

void setup(){
  Serial.begin(115200);
  pinMode(BUZZER, OUTPUT); pinMode(LED, OUTPUT);
  pinMode(CANCEL, INPUT_PULLUP);
  Wire.begin(21, 22);
  if(!mpu.begin()){Serial.println("MPU6050 not detected."); while(true) delay(1000);}
  GPS.begin(9600, SERIAL_8N1, 16, 17);
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  while(WiFi.status()!=WL_CONNECTED){delay(300); Serial.print(".");}
  Serial.println("\nWi-Fi connected."); alertOff();
}

void loop(){
  sensors_event_t a, g, temp; mpu.getEvent(&a, &g, &temp); readGPS();
  float impact=magnitudeG(a.acceleration.x, a.acceleration.y, a.acceleration.z);
  float tilt=tiltDeg(a.acceleration.x, a.acceleration.y, a.acceleration.z);

  if(state==NORMAL){
    if(impact>=CONFIRMED_G){state=ACCIDENT; alertOn(); publishEvent("ACCIDENT", impact, tilt);}
    else if(impact>=WARNING_G){state=WARNING; warningStarted=millis(); alertOn();}
  }else if(state==WARNING){
    if(digitalRead(CANCEL)==LOW){
      publishEvent("CANCELLED", impact, tilt); alertOff(); state=NORMAL; delay(300);
    }else if(millis()-warningStarted>=WARNING_TIME){
      bool confirmed=tilt>=TILT_LIMIT;
      publishEvent(confirmed?"ACCIDENT":"NO_ACCIDENT", impact, tilt);
      state=confirmed?ACCIDENT:NORMAL; if(!confirmed) alertOff();
    }
  }else if(state==ACCIDENT){
    if(digitalRead(CANCEL)==LOW){
      publishEvent("ACKNOWLEDGED", impact, tilt); alertOff(); state=NORMAL; delay(300);
    }
  }
  Serial.printf("G=%2f Tilt=%1f State=%d\n", impact, tilt, state);
  delay(100);
}

```

3. Web Dashboard Source

The dashboard displays status, impact, tilt, GPS coordinates and update time. Replace Firebase placeholders with the project's real configuration before use.

```

<!doctype html>
<html>
<head><meta charset="utf-8"><meta name="viewport" content="width=device-width, initial-scale=1">
<title>Smart Vehicle Safety</title>
<style>
body{font-family:Arial;margin:0;background:#f4f7fb;color:#182230}
nav{background:#17365d;color:#fff;padding:15px}
main{max-width:900px;margin:25px auto;padding:0 16px}
.card{background:#fff;padding:18px;margin:12px 0;border-radius:12px;box-shadow:0 2px 10px rgba(0,0,0,.08)}
.value{font-size:25px;font-weight:bold}
</style></head>
<body><nav><b>Smart Vehicle Safety</b></nav><main>
<h1>Vehicle Dashboard</h1>
<div class="card">Status: <span id="status" class="value">Loading...</span></div>
<div class="card">Impact: <span id="impact">--</span> G<br>
Tilt: <span id="tilt">--</span> degrees<br>
GPS: <span id="gps">--</span><br>Updated: <span id="time">--</span></div>
<div class="card"><a id="map" target="_blank" href="#">Open location in map</a></div>
</main>
<script type="module">
import {initializeApp} from "https://www.gstatic.com/firebasejs/10.12.5/firebase-app.js";
import {getDatabase, ref, onValue} from "https://www.gstatic.com/firebasejs/10.12.5/firebase-database.js";
const firebaseConfig={
  apiKey:"YOUR_API_KEY", authDomain:"YOUR_PROJECT.firebaseio.com",
  databaseURL:"https://YOUR_PROJECT-default-rtdb.firebaseio.com",
  projectId:"YOUR_PROJECT", storageBucket:"YOUR_PROJECT.appspot.com",
  messagingSenderId:"YOUR_SENDER_ID", appId:"YOUR_APP_ID";
const app=initializeApp(firebaseConfig), db=getDatabase(app);
onValue(ref(db, "vehicles/DEVICE_ID"), snap=>{
  const d=snap.val()||{};
  status.textContent=d.status||"UNKNOWN"; impact.textContent=d.impactG?"--";
  tilt.textContent=d.tiltDeg?"--";
  if(d.gpsValid){let lat=Number(d.latitude), lon=Number(d.longitude);
  gps.textContent=`${lat.toFixed(6)}, ${lon.toFixed(6)}`;
  map.href=`https://www.google.com/maps?q=${lat},${lon}`;
  }else gps.textContent="No valid fix";
  time.textContent=d.timestamp?new Date(d.timestamp).toLocaleString():"--";
});
</script></body></html>

```

4. Firebase Security Rules

```
{
  "rules": {
    "vehicles": {"$deviceId": {"read": "auth != null", "write": "auth != null"}},
    "events": {"$eventId": {"read": "auth != null", "write": "auth != null"}},
    "emergencyContacts": {"read": "auth != null", "write": "auth != null"}
  }
}
```

5. GitHub Repository Structure

```
smart-vehicle-safety/
├── README.md
├── hardware/
│   ├── block-diagram.pdf
│   ├── circuit-schematic.pdf
│   ├── bom.xlsx
│   └── firmware/
│       ├── src/main.cpp
│       └── platformio.ini
├── web/
│   ├── dashboard.html
│   └── firebase/
│       ├── database.rules.json
│       └── simulation/
│           ├── diagram.json
│           └── testing/
│               ├── test-cases.csv
│               └── results.csv
├── docs/
│   ├── phase3.pdf
│   ├── phase4.pdf
│   └── phase5.pdf
```

6. PCB Fabrication and Mechanical Assembly

Submit schematic/layout screenshots, PCB fabrication/order evidence if applicable, and photographs showing the assembled hardware. Mark the IMU orientation and ensure the GPS antenna has suitable exposure.

7. Hardware and Software Integration

Integration point	Verification
IMU → ESP32	Serial values update continuously
GPS → ESP32	Valid NMEA/location fix detected
ESP32 → cloud	Event record updates when network is available
Cloud → dashboard	Realtime status/location displayed
Switch → firmware	Warning cancellation works

8. Prototype Video — 3 to 5 Minutes

Time	Demonstration
0:00–0:30	Problem and objective
0:30–1:15	Hardware and architecture
1:15–2:00	Normal operation/dashboard
2:00–2:45	Controlled warning test
2:45–3:30	Cancellation/recovery
3:30–4:15	Confirmed test + GPS/dashboard
4:15–5:00	Limitations/future work

9. GitHub and Weekly Progress Report

Submission	Required evidence
GitHub	Repository URL, README and source folders
Prototype images	Clear hardware photographs

Submission	Required evidence
Prototype video	3–5 minute demonstration
Source code	Firmware + web + configuration templates
Weekly report	Work completed, issues, fixes and next steps

10. Phase 4 Submission Checklist

Deliverable	Status
Working Prototype Version 1	■
Embedded Software	■
PCB Fabrication Status if applicable	■
Mechanical Assembly if applicable	■
Hardware Integration	■
Software Integration	■
GitHub Repository Link	■
Prototype Images	■
Prototype Video (3–5 min)	■
Source Code	■
Weekly Progress Report	■

Do not publish Wi-Fi passwords, Firebase private credentials, API tokens or personal contact data in a public GitHub repository.